

# Marks4Sure Architecture Foundation

42 practice questions with answers and explanations

✓ Correct answers are highlighted in green

## Questions 4

---

In production, you observe that simple fact-checking queries—for example, “What year was the Paris Climate Agreement signed?”—traverse all four subagents sequentially, consuming more than 40 seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What is the most effective approach to optimize for varying query complexity?

### OPTIONS

- A. Create a fast path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.
- B. Train a query-complexity classifier on labeled historical data to predict optimal subagent combinations, retraining it periodically as query patterns evolve.
- C. Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of the query requirements. ✓**
- D. Implement pattern-based routing that categorizes queries by structure—single-fact, comparative, or analytical—and maps each category to a predefined subagent combination.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

Option C allows orchestration effort to scale according to the actual evidence and processing requirements of each request. A factual lookup might require one search operation and a direct coordinator response, while comparative research may justify parallel search, document analysis, synthesis, and formal report generation.

Anthropic’s orchestrator-workers guidance describes a central model dynamically decomposing a task, choosing workers, and combining their results. This pattern is particularly appropriate when required subtasks cannot be predicted reliably in advance. Anthropic’s multi-agent research architecture similarly uses a lead agent to analyze the query and determine an appropriate research strategy.

Option A creates an inflexible two-route system and still sends every non-factual query through the most expensive workflow. Option B requires labeled training data, an externally maintained classifier, and a stable definition of the optimal agent combination. Option D is less expensive to implement but becomes brittle as query types evolve beyond predetermined categories. Dynamic coordination uses the model’s existing reasoning ability to assess source requirements, analytical depth, expected output, and uncertainty before invoking only the specialists that materially improve the result.

## Questions 5

---

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

In addition to your CI pipeline, your organization has enabled Claude's managed Code Review through the Claude GitHub App on this repository, and reviews run automatically on every pull request. Reviews average 18 findings per pull request. Developer feedback reveals three categories of unwanted noise: (1) style and formatting issues already enforced by your linter in CI, (2) findings on automatically generated template code under `src/gen/`, and (3) rendering-helper patterns that are intentional project conventions but get flagged because they resemble common anti-patterns. Only approximately four findings per pull request are genuine logic bugs.

What is the most effective way to reduce this noise while preserving the detection of genuine issues?

### OPTIONS

- A. Create a `REVIEW.md` file at the repository root containing skip rules for CI-enforced checks and generated files, together with a verification requirement that rendering-related findings cite a specific line demonstrating incorrect behavior. ✓**
- B. Add custom review instructions to a GitHub Actions workflow file, using the action's prompt parameter to suppress duplicate lint findings, ignore generated template code, and apply stricter evidence requirements to rendering-related issues.
- C. Add detailed explanations to the project's `CLAUDE.md` describing which patterns are intentional, that linting is handled separately by CI, and that the `src/gen/` directory contains automatically generated template code.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

Option A uses the dedicated control surface for managed Claude Code Review. Anthropic's Code Review documentation states that a root-level `REVIEW.md` is injected into every review agent as the highest-priority instruction block. It can define skip paths, suppress categories already enforced by CI, recalibrate severity, cap nit volume, and require source evidence before reporting particular findings. The documentation explicitly identifies generated code, linting, and verification requirements as appropriate uses.

Option B configures a self-hosted GitHub Actions workflow, but the scenario concerns the separate managed Code Review service running on Anthropic's infrastructure. Instructions in that workflow do not control the managed reviewer. Option C provides useful general project context, but `CLAUDE.md` has lower review-specific authority: managed Code Review treats violations of it primarily as nit-level findings. `REVIEW.md` is the stronger and more precise mechanism for changing what the managed service reports. The file should skip `src/gen/**`, suppress style issues already enforced by CI, and require

concrete behavioral evidence for rendering-helper warnings while continuing to report verified correctness and security defects.

# CCAR-F Report Card

CANDIDATE: [REDACTED]  
CANDIDATE ID: [REDACTED]  
REGISTRATION NUMBER: [REDACTED]  
EXAM: Claude Certified Architect Foundations

DATE: 7/24/2026  
SITE NUMBER: Proctored  
EXAM CODE: CCAR-F

This report shows the percentage of items in each section you answered correctly for the Claude Certified Architect Foundation

PASSING SCORE: 720

YOUR SCORE: 921

GRADE: Pas

| TEST OBJECTIVES                                                                                                                                                                      | PERCENT CORRECT |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Design orchestration-layer safeguards ensuring every agent session ends with a completed resolution or human escalation, regardless of how the agentic loop terminates.              | 100%            |
| Design structured handoff packages that preserve accumulated context, findings, and authorization state when transferring control between agent steps or to a human operator.        | 100%            |
| Apply session resumption techniques—including targeted re-analysis of changed files and context injection—to restore agent state accurately without repeating prior work.            | 83%             |
| Decompose complex tasks into dynamically generated subtasks that adapt as new information is discovered, rather than executing a fixed sequence regardless of intermediate findings. | 100%            |
| Apply PreToolUse and PostToolUse hook patterns to enforce business rules and policy constraints not delegated to model discretion.                                                   | 100%            |
| Explain how the agentic loop uses model responses and stop_reason signals to decide whether to continue tool execution or terminate and return a final response.                     | 100%            |

## Questions 6

---

Your CI pipeline performs security-focused code reviews on approximately 50 pull requests daily, currently costing \$150 per day through the synchronous API. Reviews are non-blocking—developers merge after tests pass and address findings in follow-up commits. You are evaluating the Message Batches API because it offers a 50% cost reduction. What factor most determines whether batch processing is appropriate for this use case?

### OPTIONS

- A. Whether your result-processing system can handle reviews arriving in a different order from the order in which they were submitted.
- B. Whether each review can be structured as a single request without multi-turn refinement.
- C. Whether review feedback arriving up to 24 hours after pull-request creation remains actionable. ✓**
- D. Whether reducing per-review latency from 30–60 seconds to near-instantaneous delivery matters to your workflow.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

The decisive trade-off is whether the workflow can tolerate asynchronous completion. Anthropic's Message Batches API documentation states that batch requests receive a 50% discount and that most batches finish within one hour, but processing can continue for as long as 24 hours. Therefore, the review process is suitable for batching only if potentially delayed feedback remains useful.

Option C directly tests that operational requirement. The reviews are already non-blocking, which makes batching promising, but non-blocking does not automatically mean that feedback delivered many hours later still has value. If developers have already merged and moved to unrelated work, delayed security findings may increase remediation cost or remain unaddressed.

Option A is manageable through each request's unique `custom_id`, which allows results to be correlated regardless of return order. Option B is incorrect because batch requests can contain supported multi-turn conversation histories and tool-use configurations; each request is processed independently, but it need not represent a single conversational turn. Option D reverses the relevant consideration: batch processing is not intended to provide near-instantaneous responses. The real question is whether the cost saving justifies its longer and nondeterministic completion time.

## Questions 7

---

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web-search and document-analysis agents did not find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

### OPTIONS

- A. Have the coordinator evaluate the synthesis output for gaps, then redelegate targeted queries to the web-search and document-analysis agents before invoking synthesis again.**  
✓
- B. Have the report-generation agent identify unanswered research questions so users understand the limitations of the final output.
- C. Increase the initial breadth of queries sent to the web-search and document-analysis agents to reduce the probability of missing relevant information.
- D. Give the synthesis agent direct access to web-search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

Option A implements an iterative orchestrator-worker loop in which synthesis is treated as an evaluation checkpoint rather than an irreversible transition to report generation. When synthesis identifies missing evidence, the coordinator can formulate focused follow-up tasks, send them to the agents with the appropriate tools, and repeat synthesis after receiving the additional findings.

Anthropic's multi-agent research architecture follows this pattern: the lead agent synthesizes subagent findings and determines whether further research is required. If gaps remain, it creates additional subagents or refines the research strategy. This preserves specialization and centralized control.

Option B makes the incompleteness visible but does not improve research coverage. Option C may increase initial cost without guaranteeing that the specific gaps discovered during synthesis will be addressed. Option D weakens separation of concerns by giving the synthesis agent search capabilities that belong to the research specialists. Targeted redelegation is more efficient because the second research round is informed by concrete deficiencies rather than speculative breadth. It also allows the coordinator to track completeness explicitly before authorizing final report generation.

## Questions 8

---

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You've asked Claude Code to build a PDF report generation feature. The initial implementation queries the database correctly, but the output has formatting issues: table columns are too narrow causing content truncation, dates display without proper formatting, and page break handling is incorrect. You've noticed these issues interact—changing column widths affects how dates render, and page breaks depend on content height.

What's the most effective approach for iterating toward a working solution?

### OPTIONS

- A. Start fresh with a detailed prompt specifying all formatting requirements upfront.
- B. Provide all three issues in a single detailed message with exact specifications for each, allowing Claude to address them together in one update.
- C. Address the column width issue first with specific measurements, verify it works, then fix date formatting within the corrected columns, then adjust page breaks—testing after each change. ✓**
- D. Show Claude an example of a correctly formatted report and ask it to match that output, rather than listing the specific technical issues.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

The defects are coupled, so changing all three simultaneously would make it difficult to determine which modification caused an improvement or regression. Option C establishes a controlled sequence: correct the foundational column geometry, verify the resulting layout, format dates within the stabilized columns, and finally tune page breaks using the resulting content heights.

Anthropic recommends tight feedback loops and early course correction. It also advises supplying Claude with an executable or observable verification mechanism, such as a test, build result, generated fixture, or screenshot comparison. Claude can then make a focused change, inspect the output, and iterate until that specific condition is satisfied. ( <https://code.claude.com/docs/en/best-practices> )

Option A discards useful context from the functioning database implementation. Option B changes multiple interacting variables in one pass, making failures harder to isolate. Option D provides a useful visual target but does not replace precise technical constraints or incremental verification.

Each stage should have explicit acceptance criteria—for example, minimum column widths, expected date strings, and page-break fixtures using short and long content. Once a stage passes, its test becomes a regression guard for subsequent changes.

Official references/topics: Incremental Refinement; Tight Feedback Loops; Observable Verification; Regression Control.

## Questions 9

---

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools ( `get_customer` , `lookup_order` , `process_refund` , `escalate_to_human` ). Your target is 80%+ first-contact resolution while knowing when to escalate.

The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened.

What's the most likely cause of this behavior?

### OPTIONS

- A. The prompt lacks instructions telling Claude to remember information across multiple exchanges.
- B. The conversation history isn't being passed in subsequent API requests. ✓**
- C. The verification tool is clearing the agent's internal state after each successful validation step.
- D. Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

The Claude Messages API is stateless. Each API request must include the conversation history that Claude requires for the current response. If the application sends only the customer's third verification answer, Claude does not automatically retrieve the earlier turns containing the customer's name and previous answers. From the model's perspective, that information is absent.

Anthropic's Messages API documentation states that applications must send the full conversational history to build a multi-turn interaction. The correct implementation appends each user message and assistant response to the messages array and resends the accumulated sequence on every subsequent request. ( <https://platform.claude.com/docs/en/build-with-claude/working-with-messages> )

Option A cannot restore information that was omitted from the request. Instructions to "remember" do not create server-side conversational state. Option C describes application-specific behavior for which the scenario provides no evidence. Option D is factually incorrect: Claude does not have a default two-turn retention limit. Its effective conversational awareness depends on the messages provided and the model's context window.

For a verification workflow, the application should also persist structured verification state separately from prose history. That state may include completed checks, pending checks, attempt counts, and a verified customer identifier. This improves reliability while the conversational transcript preserves the natural interaction.

Official references/topics: Stateless Messages API, multi-turn history construction, structured workflow state, conversational continuity.

## Questions 10

---

You have configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization—the synthesis agent attempts web searches, and the report generator tries to analyze documents. What is the primary cause of this poor tool-selection behavior?

### OPTIONS

- A. The agents' role descriptions in their system prompts conflict with having access to tools outside those roles. ✓**
- B. The tool definitions consume too much context-window space, leaving insufficient room for task content.
- C. The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks.
- D. Choosing from 18 tools instead of four or five relevant tools increases decision complexity beyond reliable selection thresholds.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

Option A identifies the direct configuration contradiction. Each system prompt defines a specialized responsibility, but the corresponding tool allowlist presents unrelated actions as available capabilities. The synthesis agent may therefore interpret web searching as an acceptable way to fill evidence gaps, while the report generator may perform document analysis instead of limiting itself to report production.

Anthropic's subagent documentation defines subagents as focused workers with their own prompts and specific tool access. Its tool-definition guidance also recommends reducing selection ambiguity by presenting clear, relevant tools. The appropriate correction is least-capability configuration: expose search tools to the search agent, document tools to the analyzer, synthesis utilities to the synthesizer, and formatting or output tools to the report generator.

Option B may affect token efficiency but does not explain the role-specific misuse pattern. Option C concerns delegation, whereas the improper calls occur after successful delegation. Option D is tempting, but Anthropic does not define four or five tools as a universal reliability threshold; 18 tools alone does not prove threshold failure. The decisive defect is misalignment between declared roles and permitted capabilities.

## Questions 11

---

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated reviewer uses a single prompt covering security issues, API design, and business-logic correctness. Your evaluation suite shows strong recall for API-design findings at 82% but poor recall for business-logic edge cases in quiz scoring at 34%. When you add few-shot examples of logic bugs to the prompt, logic recall improves to 41%, but API-design recall drops to 68%.

How should you address this trade-off to improve detection across both categories?

### OPTIONS

- A. Split the review into separate focused prompts—one for security and API design and another for business logic—each with dedicated examples, and then consolidate the findings before posting. ✓**
- B. Replace the few-shot examples with a detailed checklist of specific logic edge cases to verify, such as division by zero in score calculations and boundary conditions in grading thresholds.
- C. Upgrade to a more capable model tier because its stronger reasoning will handle both concern types in one prompt and eliminate the recall trade-off.
- D. Provide the full repository as context instead of only the changed files and surrounding code, giving the model deeper visibility into business-logic patterns.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

Option A separates competing objectives so each review call can devote its attention and examples to a coherent concern. Anthropic's Building Effective AI Agents describes sectioning as a parallelization pattern in which independent aspects are handled by separate model calls and then aggregated. It specifically notes that models generally perform better on complex tasks when each consideration receives focused attention. The evaluation results demonstrate prompt interference: optimizing business-logic detection reduces API-design recall.

A security and API-design reviewer can use examples, terminology, and evidence criteria appropriate to interfaces and vulnerabilities. A business-logic reviewer can focus on state transitions, arithmetic boundaries, invariants, and domain-specific edge cases. Their structured findings can then be deduplicated and ranked before posting. Option B may improve only the weak category and can overfit to known quiz-scoring failures. Option C assumes that model capacity alone removes prompt interference without evaluation evidence. Option D adds substantial unrelated context and does not isolate the competing review objectives. The split prompts should be evaluated independently and together so consolidation does not reintroduce omissions or duplicate findings.

## Questions 12

---

Your pipeline reviews approximately 200 database-migration scripts daily using the Message Batches API. Each request includes a shared 8,000-token system prompt containing migration-review guidelines and schema documentation, followed by an individual migration script. You added `cache_control` breakpoints to the shared system prompt in every request, but monitoring shows cache-hit rates of only 32%, with misses concentrated among requests processed later in the batch window. Which change addresses the root cause without adding sequential-processing latency?

### OPTIONS

- A. Split the 200 requests into ten sequential batches of 20, submitting each batch only after the previous batch completes.
- B. Add cache-prewarming requests with `max_tokens: 0` at the beginning of every batch.
- C. Move the `cache_control` breakpoint from the shared system prompt to each migration script so similar code patterns can be reused.
- D. Configure the cache breakpoints to use the extended one-hour TTL instead of the default five-minute TTL. ✓**

### ANSWER

✓ **Correct answer: D**

### EXPLANATION

Option D directly addresses cache entries expiring before later batch requests are processed. Anthropic's batch-processing documentation specifically notes that Message Batches can take longer than five minutes and recommends the one-hour prompt-cache duration for batches containing shared context. The prompt-caching documentation confirms that the default TTL is five minutes and that `"ttl": "1h"` creates an extended entry.

Option A may improve cache locality, but it introduces the sequential-processing latency explicitly prohibited by the requirement. Option B seeds the cache initially, but the prewarmed entry still expires after five minutes unless the TTL is extended; it therefore does not solve misses among requests scheduled later. Option C places the breakpoint on request-specific content, defeating reuse of the stable 8,000-token prefix and attempting to cache scripts that are not byte-for-byte identical.

The system should retain the breakpoint at the end of the shared system content and apply the one-hour TTL there. Cache-usage fields should then be monitored to confirm that `cache_read_input_tokens` rises and later batch requests reuse the intended prefix.

## Questions 13

---

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your infrastructure-as-code repository includes Terraform modules ( /terraform/ ), Kubernetes manifests ( /kubernetes/ ), and CI/CD pipeline scripts ( /pipelines/ ). Each requires different conventions, but your single root CLAUDE.md has grown to 500+ lines. When developers work on Kubernetes files, Terraform-specific rules load into context unnecessarily, consuming tokens.

What is the best approach to reorganize so only relevant guidance loads when editing specific file types?

### OPTIONS

- A. **Create files in .claude/rules/ with YAML frontmatter path-scoping (e.g., paths: [ "terraform/\*\*/\*.tf" ] ), loading rules only when editing matching files. ✓**
- B. Restructure the root CLAUDE.md into clearly labeled sections with headers (e.g., "## Terraform Conventions"), improving organization and readability.
- C. Split content into subdirectory CLAUDE.md files ( /terraform/CLAUDE.md , /kubernetes/CLAUDE.md ), so Claude loads directory-specific guidance.
- D. Keep the root CLAUDE.md and use @path/to/import syntax to modularly include tool-specific guidance files from separate documents.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

Path-scoped rules directly satisfy the requirement that instructions load only when Claude works with matching files. Each infrastructure domain can have a separate Markdown file under .claude/rules/ , with YAML paths frontmatter targeting Terraform files, Kubernetes manifests, or pipeline configuration.

Anthropic documents that .claude/rules/ keeps project guidance modular and supports conditional loading based on glob patterns. Rules with a paths field apply only when Claude works with matching files, reducing irrelevant context and token consumption. Rules without path frontmatter load unconditionally. ( <https://code.claude.com/docs/en/memory> )

Option B improves readability but leaves all 500-plus lines in every session. Option C can provide directory-specific CLAUDE.md guidance, but path-scoped rules are more precise when conventions depend on extensions or patterns spanning multiple directories. Option D still requires manual imports and does not provide the documented conditional rule-loading mechanism.

A suitable design would use rules such as terraform.md scoped to terraform/\*\*/\*.tf , kubernetes.md scoped to kubernetes/\*\*/\*.{yaml,yml} , and pipelines.md scoped to the pipeline directory. Broad project-wide commands and repository etiquette can remain in the root CLAUDE.md.

Official references/topics: [.claude/rules/](#) ; Path-Specific Rules; YAML Frontmatter; Context-Efficient Instruction Loading.

## Questions 14

---

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks the agent to find all callers of a function before removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module).

What exploration strategy will most reliably identify all callers?

### OPTIONS

- A. Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function.
- B. Use Grep to search for the function's original name across the codebase.
- C. Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase. ✓**
- D. Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

The search must first establish the function's complete naming surface. Reading the core definition and every wrapper or re-export reveals aliases such as `computeOrderTax`, names introduced through intermediate modules, and potentially different import paths. Once those names are known, the agent can search the entire codebase for each exposed identifier and inspect the resulting call sites.

Claude Code's Grep tool searches patterns inside file contents, whereas Read provides the surrounding implementation needed to determine whether a wrapper delegates to, renames, or conditionally invokes the original function. These tools are complementary: Read establishes semantic identity; Grep provides broad reference discovery. ( [https://code.claude.com/docs/en/tools-reference?utm\\_source=chatgpt.com](https://code.claude.com/docs/en/tools-reference?utm_source=chatgpt.com) )

Option B misses every caller using a wrapper alias. Option A may find importing files, but import-level discovery is indirect and can miss re-exports, namespace imports, dependency injection, or calls made through a locally renamed symbol. It also requires unnecessary manual inspection of every consumer of the module. Option D relies on documentation, which may be incomplete or stale and cannot prove that all executable references have been found.

Before removal, the agent should also inspect tests, dynamic registrations, configuration-driven references, and generated code where relevant. The defining requirement is to map the alias chain first and then search all discovered public names.

Official references/topics: Read and Grep Tool Behavior; Symbol and Alias Discovery; Codebase Reference Tracing.

## Questions 15

---

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Testing reveals that when source documents are missing certain specifications, the model fabricates plausible-sounding values to satisfy your schema's required fields. For example, a document mentioning only dimensions receives a fabricated "weight: 2.3 kg" in the extraction output.

What schema design change most effectively addresses this hallucination behavior?

### OPTIONS

- A. Add explicit instructions to the prompt stating "only extract information explicitly stated in the document; use placeholder text for missing values."
- B. Change fields that may not exist in source documents from required to optional, allowing the model to omit them. ✓**
- C. Add a "confidence" field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions.
- D. Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

The schema is creating a structural incentive for fabrication. When a field is declared required, the output must contain a value even when the source document contains no corresponding evidence. Structured Outputs can guarantee that Claude's response conforms to a JSON Schema, but schema conformance does not establish that every generated value is factually supported. Anthropic's documentation shows that the required array determines which properties must be present; therefore, source-dependent properties that may legitimately be absent should not be included as required fields. ( <https://platform.claude.com/docs/en/build-with-claude/structured-outputs> )

Option B corrects the problem at the contract level. Claude can omit the unavailable property rather than inventing content merely to produce valid JSON. A nullable representation could also be used when downstream systems require a stable key set, but forcing an unsupported non-null value is architecturally unsound.

Option A still requires placeholder generation and does not resolve the mismatch between the schema and available evidence. Option C relies on model-generated confidence, which is not a substitute for grounding. Option D is a useful secondary control, but it does not constitute the requested schema-design change. Anthropic recommends allowing uncertainty and requiring factual claims to be grounded in the provided material. ( <https://docs.anthropic.com/en/docs/test-and-evaluate/strengthen-guardrails/reduce-hallucinations> )

Official references/topics: Structured Outputs—JSON Schema design; Reduce Hallucinations—allowing uncertainty and grounding claims.

## Questions 16

---

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You've asked Claude to write a data migration script, but the initial output doesn't correctly handle records with null values in required fields.

What's the most effective way to iterate toward a working solution?

### OPTIONS

- A. Add "think harder about edge cases" to your prompt and request a complete rewrite of the migration logic.
- B. Manually edit the generated code to fix the null handling, then continue working with Claude on other parts.
- C. Describe the null value problem in detail and ask Claude to regenerate the entire script with improved edge case handling.
- D. Provide a test case with example input containing null values and the expected output, then ask Claude to fix it. ✓**

### ANSWER

✓ **Correct answer: D**

### EXPLANATION

A concrete test case converts an imprecise correction into an executable success criterion. The input demonstrates the failing null-value condition, while the expected output defines exactly how the migration must handle it. Claude can modify the implementation, run the test, inspect the result, and continue iterating until the behavior passes.

Anthropic recommends giving Claude a verification mechanism such as a test suite, build result, linter, fixture comparison, or screenshot. Without such a mechanism, Claude can only judge that its output appears correct. A pass-or-fail test closes the feedback loop by allowing Claude to perform the work, evaluate the result, and refine it using objective evidence. ( <https://code.claude.com/docs/en/best-practices> )

Option A provides no additional specification and may generate a different but still incorrect implementation. Option B fixes the immediate defect manually but fails to use Claude's iterative capabilities or establish a regression check. Option C supplies more description, but regenerating the entire script increases the risk of altering already-correct behavior and still lacks objective validation.

The best workflow is to add the failing case to the migration test suite, request the smallest necessary correction, run the relevant tests, and retain that test permanently to prevent the null-handling defect from recurring.

Official references/topics: Executable verification, test-driven iteration, regression protection, tight feedback loops.

## Questions 17

---

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

After implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still contain empty arrays or null values for required fields such as citations and methodology. Spot-checking reveals that the source documents contain this information, but in varied formats—inline citations versus bibliographies, and methodology sections versus details embedded in introductions.

What is the most effective way to address these failures?

### OPTIONS

- A. Implement retry logic that resends requests when validation detects empty required fields.
- B. Add few-shot examples demonstrating extractions from documents with varied structures, showing how to identify citations in different formats and locate methodology details across section types. ✓**
- C. Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract them.
- D. Modify the schema to make citations and methodology optional, and flag incomplete records for manual review instead of failing validation.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

Option B targets the remaining failure mode: semantic recognition across heterogeneous document structures. Strict schemas eliminate malformed JSON and can guarantee that tool inputs conform to declared types, but they cannot force Claude to locate evidence that appears under unfamiliar headings or in atypical sections. Anthropic's prompting guidance says that a few relevant, diverse, structured examples are among the most reliable ways to improve accuracy and consistency. Examples should therefore show inline citations, reference lists, numbered bibliographies, methodology sections, and methods embedded in introductions, each paired with the correct extracted structure. This teaches the intended evidence-location and granularity rules rather than merely repeating the same request. Option A retries an unchanged prompt and can reproduce the same omission. Option C introduces brittle regex rules that may miss nonstandard citations and mistake keyword mentions for methodology content. Option D suppresses validation failures by weakening the contract, but it does not improve extraction and would convert recoverable omissions into incomplete records. The examples should be drawn from real failure cases, evaluated on a held-out set, and expanded when monitoring reveals new layouts. Schema constraints and few-shot coverage solve different layers of reliability and should be used together.

## Questions 18

---

Your pipeline runs:

```
PROMPT= " You are a code reviewer. "
```

```
PROMPT= " $PROMPT Analyze the provided diff "
```

```
PROMPT= " $PROMPT for bugs, security issues, "
```

```
PROMPT= " $PROMPT and style violations. "
```

```
claude -p \
```

```
--dangerously-skip-permissions \
```

```
--system-prompt " $PROMPT " < diff.txt
```

The reviews complete and return feedback, but Claude comments only on the piped diff—it never reads surrounding files in the checked-out repository to understand broader context, even when the diff modifies a function called by many other modules. Which change to the invocation will cause Claude to read related repository files while still applying your custom review instructions?

### OPTIONS

- A. Keep `--system-prompt` and add `--allowedTools " Read, Glob, Grep "` because non-interactive `-p` mode otherwise disables filesystem tools.
- B. Replace `--system-prompt` with `--append-system-prompt` so the review instructions are added to Claude Code's default prompt instead of overwriting its built-in file-reading and code-navigation guidance. ✓**
- C. Remove `--system-prompt` entirely and place the review instructions in a root-level `CLAUDE.md` because `--system-prompt` is incompatible with tool use under `-p`.
- D. Stop piping the diff through standard input and embed it in the prompt string so Claude Code treats the invocation as an agentic session rather than a stream-processing operation.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

The `--system-prompt` flag replaces Claude Code's complete default system prompt. Although this does not technically remove the available tools, it discards the default coding-agent guidance that tells Claude how to gather repository context, navigate code, use tools, and verify findings. The replacement prompt says only to analyze the supplied diff, so the observed diff-only behaviour is consistent with the configured instructions.

Option B preserves the standard Claude Code identity and tool guidance while adding the custom review criteria. The Claude Code CLI reference states that `--append-system-prompt` appends instructions to the default prompt, whereas `--system-prompt` replaces it. Anthropic specifically

recommends appending when Claude should remain a coding assistant that follows additional per-invocation rules.

Option A is incorrect because print mode does not inherently disable Read, Glob, or Grep; `--allowedTools` restricts or pre-authorizes tools but does not restore overwritten behavioural guidance. Option C is incorrect because replacement system prompts remain compatible with tools, although the replacement must supply suitable tool instructions. Option D changes where the diff appears but does not restore Claude Code's default code-navigation behaviour. Appending the review instructions directly addresses the configuration error.

## Questions 19

---

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're implementing a caching layer for API responses to speed up the /products endpoint. You have a rough idea—Redis with a 5-minute TTL—but you're new to production caching and aren't sure what other considerations a robust implementation requires.

What's the most effective way to start your iterative workflow?

### OPTIONS

- A. Ask Claude to interview you about the caching requirements before implementing, surfacing considerations like invalidation strategies, cache layers, consistency guarantees, and failure modes. ✓**
- B. Use plan mode to analyze the current /products endpoint implementation, then provide your caching requirements once Claude explains how the existing code is structured.
- C. Start with a minimal request: "Add Redis caching to /products with 5-minute TTL." Add features and fix issues through follow-up prompts as problems surface during testing.
- D. Write a specification with your known requirements and "TBD" markers for uncertain areas, having Claude propose solutions for each TBD as it implements.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

The primary risk is not implementation difficulty but incomplete requirements. Production caching introduces decisions involving invalidation, stale-data tolerance, cache keys, tenant boundaries, serialization, stampede prevention, failure behavior, observability, deployment topology, and consistency expectations. Implementing Redis with a five-minute TTL before resolving these questions can produce a technically functional but operationally unsafe design.

Anthropic recommends having Claude interview the user before beginning a larger feature when important requirements remain uncertain. The AskUserQuestion workflow is intended to surface technical implementation concerns, edge cases, trade-offs, and assumptions the user may not have considered. Anthropic further recommends converting the resulting answers into a self-contained specification with explicit scope and an end-to-end verification step. (<https://code.claude.com/docs/en/best-practices>)

Option B provides useful codebase context but postpones requirement discovery. Option C creates avoidable rework by allowing architecture to emerge from production failures. Option D documents uncertainty but delegates unresolved design choices during implementation, when they may already constrain the code.

After the interview produces a caching specification, Claude can enter plan mode to inspect the endpoint and map those requirements onto the existing architecture before implementation.

Official references/topics: Requirements Interviewing; AskUserQuestion; Specification Development; Edge-Case and Trade-Off Discovery.

## Questions 20

---

Your automated review generates many findings per pull request, but developer feedback shows that roughly half are dismissed as “not worth addressing.” Analysis reveals that dismissed findings are often technically accurate but involve minor style preferences or patterns that are acceptable in your codebase. Before adding infrastructure complexity, what prompt-design change could most effectively reduce dismissals while maintaining the detection of genuine issues?

### OPTIONS

- A. Add explicit criteria defining which issues to report, such as bugs and security defects, and which issues to skip, such as minor style preferences and accepted local patterns. ✓**
- B. Implement a secondary classification model that filters Claude’s findings according to predicted developer acceptance.
- C. Ask Claude to rate every finding’s confidence from 1 to 10 and include only findings rated 8 or higher.
- D. Append instructions telling Claude to “only report findings you are highly confident are genuine problems.”

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

The problem is an undefined relevance threshold. Claude is finding technically valid observations, but the prompt does not clearly distinguish actionable defects from acceptable conventions and low-value style preferences. Option A establishes explicit positive and negative reporting criteria, allowing the model to apply the team’s actual definition of a useful finding during generation.

Anthropic’s prompt-engineering guidance emphasizes clear, specific instructions and well-defined success criteria. The managed Code Review documentation similarly recommends defining skipped categories, generated paths, severity rules, and evidence requirements. Reporting “bugs affecting correctness or security” while excluding “formatting handled by CI and documented local conventions” is substantially more precise than asking the model to be generally conservative.

Option B adds cost, latency, another probabilistic decision, and a new evaluation surface before improving the original prompt. Option C confuses confidence with importance: Claude may be highly confident about a trivial style observation. Option D is dangerously broad because it can suppress legitimate but uncertain bugs, reducing recall. Explicit relevance criteria address the demonstrated failure directly while preserving the model’s ability to investigate and report genuine correctness or security problems.

## Questions 21

---

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Your multi-agent research pipeline crashes after processing 12 of 28 documents. The web-search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun identifying patterns. You need to resume processing without repeating work or losing fidelity in the prior findings.

What state-management approach best balances information fidelity with context efficiency when restoring agent state?

### OPTIONS

- A. Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings.
- B. Have each agent persist a structured export to a known location. On resumption, the coordinator loads the manifest and injects relevant state into agent prompts. ✓**
- C. Have each agent maintain its own persistent state file and reload it independently at the beginning of every session.
- D. Persist the coordinator's conversation log containing all task delegations and responses, providing this log to agents when resuming.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

Option B combines lossless persistence with selective restoration. Each agent should checkpoint structured records containing completed document IDs, extracted findings, source metadata, unresolved work, validation status, and schema version. A coordinator-level manifest records where those artifacts are stored and which pipeline stages completed. On resumption, the coordinator can skip the 12 completed documents and inject only the artifacts relevant to each restarted agent. Anthropic's effective context-engineering guidance recommends structured note-taking outside the context window so agents can preserve project state across resets while loading only the information needed later. A vector store, option A, is useful for relevance retrieval but can omit exact details through similarity-based selection and is unsuitable as the sole recovery record. Option C fragments ownership of recovery state and makes cross-agent consistency difficult to verify. Option D preserves an extensive trace, but replaying a complete conversation wastes context and mixes obsolete instructions, intermediate reasoning, and redundant outputs. Structured checkpoints plus a manifest provide deterministic recovery, auditability, exact provenance, and efficient prompt reconstruction. Checkpoints should be written atomically and validated before the corresponding stage is marked complete.

## Questions 22

---

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools ( `get_customer` , `lookup_order` , `process_refund` , `escalate_to_human` ). Your target is 80%+ first-contact resolution while knowing when to escalate.

You're implementing the escalation logic for when the agent should call `escalate_to_human` . Your team proposes four different approaches for triggering escalation.

Which approach will most reliably identify cases that genuinely require human intervention?

### OPTIONS

- A. Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls.
- B. Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress. ✓**
- C. Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.
- D. Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and triggers escalation when the frustration score exceeds a configured threshold.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

Option B identifies escalation through direct operational criteria rather than indirect proxies. A customer's explicit request for a person is unambiguous. A required policy exception defines an authorization boundary. An inability to make meaningful progress identifies a case where continued autonomous execution is no longer productive.

Anthropic describes agents as systems that act, observe results, adjust, and continue until the task is completed or human input is required. Its trustworthy-agent guidance also emphasizes that agents must recognize when to pause rather than pushing through uncertainty or decisions that only a human can settle. ( <https://www.anthropic.com/research/trustworthy-agents> )

Option A is too rigid for high-ambiguity support cases. Maintaining exhaustive mappings for every issue, product, and customer segment recreates the brittleness that agentic reasoning is intended to avoid. Option C treats repeated tool use as a substitute for semantic progress; one definitive authorization failure may require immediate escalation, while several legitimate diagnostic calls may not. Option D confuses emotional tone with operational necessity. A calm customer may require a policy exception, while a frustrated customer may still have a straightforward automated resolution.

The escalation criteria should be encoded in the system instructions and supported by evaluations covering explicit requests, authorization limits, unresolved ambiguity, repeated non-progress, and successful self-service cases.

Official references/topics: Human-control checkpoints, escalation criteria, authorization boundaries, progress-aware agents.

## Questions 23

---

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools ( `get_customer` , `lookup_order` , `process_refund` , `escalate_to_human` ). Your target is 80%+ first-contact resolution while knowing when to escalate.

During a billing dispute resolution, your agent successfully retrieves customer info via `get_customer` and order details via `lookup_order` , but when attempting to call `process_refund` , the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure.

What approach best balances first-contact resolution with appropriate error handling?

### OPTIONS

- A. Implement automatic retries with exponential backoff for `process_refund` , keeping the conversation open until the refund is successfully processed.
- B. Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically.
- C. Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later. ✓**
- D. Escalate immediately to a human agent since the refund action cannot be completed.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

First-contact resolution does not require pretending that every backend operation succeeded. The agent can still resolve the informational portion of the interaction by explaining the charge and confirming eligibility using the successfully retrieved customer and order data. It must then distinguish that verified conclusion from the uncompleted refund transaction.

Anthropic's tool-error guidance states that Claude should receive the failure information so it can retry, request clarification, or explain the limitation. A tool timeout must therefore be surfaced honestly rather than converted into an unsupported success claim. ( [https://platform.claude.com/docs/en/agents-and-tools/tool-use/build-a-tool-using-agent?utm\\_source=chatgpt.com](https://platform.claude.com/docs/en/agents-and-tools/tool-use/build-a-tool-using-agent?utm_source=chatgpt.com) ) Anthropic also emphasizes transparent, simple agent designs and carefully constructed tool interfaces, which support explicit disclosure of tool failure and controlled escalation. ( <https://www.anthropic.com/engineering/building-effective-agents> )

Option A can create an unbounded or excessively long interaction; retries should be limited and conditioned on retryability. Option B falsely represents an incomplete financial operation as completed. Option D discards the useful work already performed and escalates before providing the customer with the available explanation.

Option C preserves trust, delivers the information already established, clearly states what remains incomplete, and gives the customer a practical next step through bounded retry or human escalation.

Official references/topics: Graceful tool failure, transparent customer communication, bounded retry, human escalation.

## Questions 24

---

Your code-review prompts include both implementation changes and the corresponding test file, but the review comments fail to identify untested code paths. The model correctly flags functions that have no tests at all, but it fails to recognize when conditional branches or error-handling paths within tested functions lack coverage. What is the most effective way to improve branch-level gap detection without overcomplicating the pipeline?

### OPTIONS

- A. Interleave the implementation and tests in the prompt, presenting each function immediately before its test cases.
- B. Add explicit instructions requiring Claude to enumerate every conditional branch and exception path, then verify that each path has a corresponding test assertion. ✓**
- C. Implement a two-pass pipeline in which one model call extracts all conditional branches and another cross-references them against test assertions.
- D. Include few-shot examples showing code with an uncovered branch and the corresponding review comment identifying the missing test case.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

The current prompt asks for testing analysis at too high a level. Claude recognizes the obvious absence of an entire test but has not been instructed to construct a path-level inventory. Option B turns the desired behavior into an explicit verification procedure: enumerate each condition, alternative branch, early return, exception handler, and failure path, then locate a test assertion that exercises its behavior.

Anthropic's prompting best practices emphasize clear, specific instructions and explicit sequential steps when a task requires a defined analysis process. This change keeps the existing single review call while making the missing evaluation criterion unambiguous.

Option A improves proximity between implementation and tests but does not tell Claude what coverage relationship to inspect. Option C could work, but it adds orchestration, latency, cost, and another handoff before testing whether a direct instruction solves the observed failure. Option D may improve recognition of examples resembling the demonstration, but a few cases cannot enumerate every branch structure. Explicit path enumeration generalizes across unfamiliar code and creates auditable output: each reported gap can name the uncovered condition, expected behavior, and missing assertion.

## Questions 25

---

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools ( `get_customer` , `lookup_order` , `process_refund` , `escalate_to_human` ). Your target is 80%+ first-contact resolution while knowing when to escalate.

Anthropic's tool use documentation states: "Write instructive error messages. Instead of generic errors like 'failed', include what went wrong and what Claude should try next." A billing dispute agent uses `lookup_order` , which catches all exceptions and returns a `tool_result` with `is_error: true` and the message "Tool execution failed". Monitoring shows two failure modes: the agent retries the identical call until hitting the turn limit, or it immediately calls `escalate_to_human` without trying alternative tools.

Which change follows the documented recommendation and gives Claude the information it needs to select the correct recovery action for each error type?

### OPTIONS

- A. Implement retry logic with exponential backoff inside each tool implementation so transient errors are resolved transparently within the tool before any failure result is surfaced to Claude in the agentic loop.
- B. Return error-type-specific messages with `is_error: true` , e.g., "Order not found—try `get_customer` to search by phone" for data errors and "Database timeout (transient)—retry should succeed" for infrastructure errors. ✓**
- C. Remove `is_error: true` and return the error details as normal tool content, so Claude reasons about the response as data rather than treating it as a flagged failure condition that biases retry behavior.
- D. Add an error classification step in the agentic loop that intercepts tool errors before Claude sees them, then routes to hardcoded retry or escalation logic.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

Option B preserves the formal failure indicator while making the returned content operationally useful. Anthropic's tool-use guidance states that when a tool fails, the application should return the error through `tool_result` content and mark it with `is_error: true` . Claude can then use that information to retry with corrected input, seek clarification, or explain the limitation. ( <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/bash-tool> )

The two example messages identify both the cause and the appropriate recovery path. "Order not found" indicates that repeating the same identifier will not help and proposes an alternative lookup method. "Database timeout" identifies a transient infrastructure condition for which a retry is reasonable. This eliminates blind repetition without removing Claude's ability to adapt.

Option A can be useful for tightly bounded low-level retries, but it does not solve permanent errors or tell the agent what happened after retries fail. Option C incorrectly disguises a failed operation as successful tool content. Option D moves semantic knowledge into hardcoded orchestration logic, making the system more rigid and duplicating information already known by the tool.

The most reliable tool interface combines the error flag, a stable error type, a retryability indicator, a concise causal explanation, and a recommended recovery action.

Official references/topics: `is_error` handling, instructive tool results, recovery guidance, resilient MCP tool design.

## Questions 26

---

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline occasionally receives responses that cannot be parsed as valid JSON, causing downstream processing failures. The current implementation prompts Claude to return JSON in the response text and then parses it.

What is the most reliable approach to ensure Claude returns valid, schema-compliant structured data?

### OPTIONS

- A. Add explicit formatting instructions to the prompt with JSON examples, emphasizing that Claude must return only valid JSON with no surrounding text.
- B. Use regular expressions to locate and extract JSON from the response text, handling cases where Claude includes explanatory text around the JSON block.
- C. Define a tool with a JSON schema specifying the expected structure, using tool use to constrain Claude's output to schema-compliant JSON. ✓**
- D. Implement a retry loop that catches JSON parsing errors and re-prompts Claude with the error details, asking it to correct the malformed output.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

Option C is the strongest choice among the listed approaches because it moves structured data generation from unconstrained response text into a schema-governed interface. Prompt-only JSON instructions, regular-expression extraction, and corrective retries are probabilistic recovery mechanisms: they may reduce failures, but none guarantees that the first response satisfies the contract. Anthropic now provides two schema-constrained mechanisms: JSON Structured Outputs through `output_config.format` and strict tool use through a tool definition with `strict: true`. Strict tool use uses grammar-constrained sampling so the tool input conforms to the declared JSON Schema, including required properties and data types. Therefore, the production implementation represented by option C should define the extraction tool's `input_schema` and enable strict mode. The application reads the `tool_use` block as the extracted record; it does not need to scrape prose or search for braces. Option A remains vulnerable to preambles or malformed output. Option B can select a JSON-looking substring but cannot establish schema validity. Option D adds latency and cost and can still fail repeatedly. The schema-constrained tool contract is consequently the most reliable architecture available in these options.

## Questions 27

---

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

When analyzing complex legal cases that cite multiple precedents, the document-analysis subagent processes each precedent sequentially. A landmark case citing 12 precedents takes more than three minutes to analyze completely.

What is the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

### OPTIONS

- A. Implement a message queue where precedent-analysis tasks are processed asynchronously by a pool of worker agents.
- B. Enable the document-analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations.
- C. Have the coordinator spawn parallel document-analysis subagents, each handling a subset of precedents, and then aggregate the results before synthesis. ✓**
- D. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

Option C parallelizes independent work while retaining centralized orchestration. Each precedent can be analyzed without waiting for the previous precedent, so the coordinator can divide the 12 cases into balanced subsets and invoke several document-analysis subagents concurrently. It then receives their final outputs, records which precedents completed or failed, and aggregates the results before synthesis. Anthropic's multi-agent research architecture uses an orchestrator-worker pattern in which the lead agent creates specialized subagents that operate in parallel and return findings for consolidation. Keeping spawning decisions at the coordinator also produces a clearer execution trace for monitoring and debugging. A generic asynchronous queue, option A, adds infrastructure but does not define how results remain associated with the correct research task. Options B and D create nested or recursive delegation, making execution paths, permissions, failures, and token consumption harder to observe. Anthropic also cautions that multi-agent systems consume substantially more tokens than ordinary interactions, so unbounded recursive decomposition is inefficient. Coordinator-controlled parallel fan-out followed by deterministic aggregation provides the latency improvement without sacrificing operational visibility.

## Questions 28

---

Production monitoring shows that the research phase takes longer than expected. Analysis reveals that the coordinator invokes the web-search subagent, waits for its response, and then invokes the document-analysis subagent. These tasks are independent; neither requires the other's output. How should you modify the system to run these subagents concurrently?

### OPTIONS

- A. Structure the coordinator to emit both Agent tool calls—for web search and document analysis—in a single response message instead of separate conversation turns. ✓**
- B. Switch both subagents from a Sonnet-tier model to a Haiku-tier model to reduce their individual execution times.
- C. Add instructions explaining the performance benefits of parallel execution and request that the coordinator invoke both subagents simultaneously.
- D. Create an asynchronous orchestration layer that launches parallel threads, each running a separate coordinator-subagent pair, and then aggregates the results.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

Option A exposes both independent invocations in the same assistant turn, allowing the Agent SDK or application tool runner to execute them concurrently. The coordinator can then receive both results together and continue with synthesis only after the independent research branches have completed.

Anthropic's parallel tool-use documentation explains that a response may contain multiple tool-use blocks. Independent, read-only operations can be executed concurrently to reduce latency, after which all corresponding tool results should be returned together. The term "Agent" is used here because current Claude Agent SDK releases renamed the earlier "Task" tool.

Option B may shorten individual execution but does not eliminate the sequential waiting pattern and could reduce research quality. Option C expresses the desired behavior but does not correct an orchestration implementation that processes only one tool call per turn. Option D introduces unnecessary coordinators, duplicated context, and substantially more complex state management. A single coordinator issuing both independent Agent calls preserves centralized monitoring and result association while removing the avoidable serial dependency. The runtime must process every returned tool call concurrently rather than stopping after the first one.

## Questions 29

---

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your pipeline includes a release-notes generation step that classifies and summarizes approximately 200 commits at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API request using a Sonnet-tier Claude model. The release notes are not needed until the following morning, providing approximately 12 hours of acceptable latency.

Your team must reduce the per-token API cost while retaining the same model, prompts, and output quality.

Which approach satisfies all these constraints?

### OPTIONS

- A. Issue the 200 Messages API requests concurrently because parallel execution reduces the per-token price.
- B. Submit the 200 requests through the Message Batches API with unique custom\_id values and retrieve the results after the batch finishes. ✓**
- C. Concatenate all 200 commit messages into one Messages API request because reducing the number of requests always reduces token costs.
- D. Replace the Sonnet-tier model with a Haiku-tier model to obtain a lower per-token price.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

Option B applies Anthropic's dedicated asynchronous bulk-processing mechanism while preserving the existing model and prompt for every commit. The Message Batches API accepts independent Messages API requests, each identified by a unique custom\_id, and charges both input and output usage at 50% of standard API prices. The approximately 12-hour latency allowance makes the release-notes workload well suited to batching because immediate results are unnecessary.

Option A may reduce wall-clock completion time, but concurrency does not alter the API's per-token price. Option C changes the task structure and risks exceeding context or output limits, mixing commit-level classifications, complicating retries, and making it harder to associate errors with individual commits. A single large request also does not automatically consume fewer tokens because the model must still process all commit content. Option D violates the requirement to retain the same model tier and output-quality profile. Batch results may arrive in an order different from submission order, so the pipeline must associate every response with its original commit through custom\_id. This provides lower cost without changing the individual review prompts. Anthropic Message Batches documentation

## Questions 30

---

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools ( `get_customer` , `lookup_order` , `process_refund` , `escalate_to_human` ). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your `process_refund` tool returns two types of errors: technical errors (“503 Service Unavailable”, “Connection timeout”) that are transient (~5% of calls), and business errors (“Order exceeds 30-day return window”, “Item already refunded”) that are permanent (~12% of calls). Monitoring shows the agent wastes 3–4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude.

What’s the most effective way to reduce wasted retries while improving customer-facing response quality?

### OPTIONS

- A. Implement automatic retry logic at the tool layer for technical errors only, passing business errors to Claude without retries.
- B. Add few-shot examples showing how to distinguish retrievable from non-retrievable errors by parsing error message text.
- C. Add a `check_refund_eligibility` tool that must be called before `process_refund` to prevent business rule violations.
- D. Return structured error responses with " `retrievable` " : `false` for business errors and a customer-friendly explanation for Claude to use. ✓**

### ANSWER

✓ **Correct answer: D**

### EXPLANATION

The error response must explicitly communicate both control semantics and user-facing meaning. Setting " `retrievable` " : `false` tells the agent that repeating the same operation cannot change the outcome. Providing a customer-friendly explanation allows Claude to respond accurately without exposing internal implementation details or inventing its own interpretation of the business rule.

Anthropic recommends returning failed tool operations with an error indicator and sufficiently informative content so Claude can decide whether to retry, request another input, or explain the limitation. Tool interfaces should provide detailed descriptions and structured parameters rather than forcing Claude to infer operational behavior from ambiguous text. ( [https://platform.claude.com/docs/en/agents-and-tools/tool-use/build-a-tool-using-agent?utm\\_source=chatgpt.com](https://platform.claude.com/docs/en/agents-and-tools/tool-use/build-a-tool-using-agent?utm_source=chatgpt.com) )

Option A appropriately limits automatic retries to technical failures, but it does not solve the stated customer-response problem unless business failures also carry clear semantics. Option B relies on

fragile parsing of human-readable messages. Option C adds latency and another tool dependency, and eligibility may still change or fail for reasons not covered by the preliminary check.

A complete schema should identify the error code, category, retryability, customer-safe explanation, and permitted next actions. Technical errors can similarly return "retriable" : true with retry guidance and a maximum-attempt policy.

Official references/topics: Structured tool errors, retryability classification, customer-safe explanations, resilient MCP contracts.

## Questions 31

---

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools ( `get_customer` , `lookup_order` , `process_refund` , `escalate_to_human` ). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your agent is handling a billing dispute. After calling `get_customer` and `lookup_order` , it identifies that the dispute involves a promotional pricing error requiring manager approval—beyond the agent’s authorization level.

How should the workflow handle this mid-process escalation?

### OPTIONS

- A. Call `escalate_to_human` , passing only the customer’s original message.
- B. Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human` . ✓**
- C. Attempt the refund with `process_refund` anyway, escalating only if the system rejects the transaction.
- D. Persist the complete conversation and tool response history to a database, then call `escalate_to_human` with a reference ID.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

A mid-process escalation should transfer the decision-ready state accumulated by the agent. The human reviewer needs the verified customer identity, relevant order information, the promotional-pricing discrepancy, the reason approval is required, and any actions already attempted. Option B preserves this information in a concise, structured handoff while avoiding unnecessary repetition of the complete raw transcript.

Anthropic’s tool-design guidance recommends returning high-signal information and stable identifiers containing only what Claude or the next workflow participant needs to determine the next action. Anthropic’s context-engineering guidance similarly advocates structured notes that preserve critical state and dependencies without retaining every redundant tool result. A structured escalation payload applies both principles and reduces handling time for the manager. ( <https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools> )

Option A discards the investigation already completed. Option C violates the agent’s authorization boundary and risks an impermissible financial action. Option D provides auditability, but a reference ID alone forces the human to reconstruct the case from an excessively broad transcript. Human control must remain meaningful when an agent encounters a decision outside its authority; the agent should pause and hand the decision back with sufficient supporting context. ( <https://www.anthropic.com/research/trustworthy-agents> )

Official references/topics: Structured agent handoffs, high-signal tool results, human-control boundaries, persistent structured state.

## Questions 32

---

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI pipeline performs security-focused code reviews on approximately 50 pull requests daily, currently costing \$150 per day using the synchronous API. Reviews are non-blocking—developers merge after tests pass and address findings in follow-up commits. You are evaluating the Message Batches API for its 50% cost reduction.

What factor most determines whether batch processing is appropriate for this use case?

### OPTIONS

- A. Whether reducing per-review latency from 30–60 seconds to near-instant matters for your workflow.
- B. Whether your result-processing system can handle reviews arriving in a different order from the order in which they were submitted.
- C. Whether review feedback arriving up to 24 hours after pull-request creation remains actionable. ✓**
- D. Whether you can structure each review as a single request without multi-turn refinement.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

Option C identifies the fundamental trade-off introduced by batch processing: lower cost in exchange for asynchronous completion and potentially substantial latency. Anthropic states that most Message Batches complete within one hour, but results may become available only when all requests finish or after 24 hours, whichever occurs first. Therefore, the workflow must remain useful even if security findings arrive considerably later than they would through synchronous requests.

The reviews are explicitly non-blocking, so batch processing can be suitable if developers can still act on delayed findings through follow-up commits. Option B describes an implementation requirement because batch results may be returned out of submission order, but Anthropic provides a deterministic solution through each request's unique `custom_id`. Option A incorrectly focuses on near-instant feedback, which batch processing does not provide. Option D is not decisive because batches support independent Messages API requests containing system prompts, tool use, and multi-turn conversation content. The primary decision is therefore whether the maximum practical feedback delay is acceptable to the development workflow. Anthropic Message Batches documentation

## Questions 33

---

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools ( `get_customer` , `lookup_order` , `process_refund` , `escalate_to_human` ). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer raises three separate issues during one session: a refund inquiry (turns 1–15), a subscription question (turns 16–30), and a payment method update (turns 31–45). At turn 48, the customer asks “What happened with my refund?” The conversation is approaching context limits.

What strategy best maintains the agent’s ability to address all issues throughout the session?

### OPTIONS

- A. Summarize earlier turns into a narrative description, preserving full message history only for the active issue.
- B. Implement sliding window context that retains the most recent 30 turns.
- C. Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues.
- D. Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer. ✓**

### ANSWER

✓ **Correct answer: D**

### EXPLANATION

The agent must preserve durable state for several concurrently open issues without keeping every conversational turn in the active context window. Option D creates a structured issue ledger containing identifiers, issue type, requested action, current status, completed steps, unresolved questions, and relevant tool outputs.

Anthropic recommends structured note-taking for long-horizon agents. In this pattern, critical state is written outside the context window and retrieved later, allowing the agent to preserve dependencies and progress across dozens of tool calls. Anthropic also describes compaction as retaining critical details while discarding redundant raw messages and historical tool output. ( <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents> )

Option A is better than retaining the entire transcript, but a narrative summary can merge separate issues or obscure exact identifiers and statuses. Option B will discard the refund conversation because it occurred outside the most recent 30 turns. Option C may recover current backend facts, but it cannot reconstruct conversational commitments, prior explanations, or why a particular action remains pending.

A structured context layer should maintain one record per issue and be updated after every material action. The active prompt can include compact summaries of all open issues and expanded details only for the issue currently being discussed.

Official references/topics: Structured note-taking, external agent memory, context compaction, multi-issue state management.

## Questions 34

---

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web-search and document-analysis agents did not find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage.

What change would most effectively improve research completeness?

### OPTIONS

- A. Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.
- B. Have the coordinator evaluate the synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again. ✓**
- C. Have the report-generation agent note which research questions could not be answered, so users understand the limitations of the final output.
- D. Give the synthesis agent direct access to web-search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

### ANSWER

✓ **Correct answer: B**

### EXPLANATION

Option B introduces an evaluator-and-refinement loop at the correct orchestration layer. The coordinator already owns the research plan and delegation decisions, so it should inspect the synthesis result against the required questions, identify coverage gaps, and issue focused follow-up assignments. Anthropic's description of its multi-agent research system follows this pattern: the lead agent synthesizes returned findings, determines whether additional research is required, and creates new subagents or refines its strategy before producing the final result. Increasing the initial query breadth, option A, may generate additional irrelevant material and cannot guarantee that unforeseen gaps will be covered. Option C merely documents the incompleteness instead of correcting it. Option D weakens role separation by giving the synthesis agent search capabilities, increasing tool complexity and bypassing the coordinator's centralized tracking. Targeted re-delegation preserves specialized responsibilities and creates an observable sequence of research, evaluation, refinement, and resynthesis. The coordinator should also maintain explicit coverage criteria and limit the number of refinement rounds so the system improves completeness without entering an uncontrolled research loop.

## Questions 35

---

Your pipeline includes a release-notes generation step that classifies and summarizes approximately 200 commits at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API call using a Sonnet-tier Claude model. The release notes are not needed until the following morning, so results have approximately 12 hours of acceptable latency. Your team needs to reduce per-token API cost for this step while keeping the same model and prompts, with no change to the model tier or output quality. Which approach satisfies all these constraints?

### OPTIONS

- A. Concatenate all 200 commit messages into a single Messages API request and have the model return all summaries in one response, because fewer requests always reduce total token cost.
- B. Issue the 200 Messages API requests in parallel using concurrent connections, because concurrency lowers the per-token price charged by the API.
- C. Submit the 200 requests to the Message Batches API with unique custom\_id values and retrieve the results after the batch finishes, which applies a 50% discount to all input and output tokens. ✓**
- D. Switch the summarization calls from the Sonnet-tier model to a Haiku-tier model to take advantage of Haiku's lower per-token rates.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

Option C preserves each commit's existing model, prompt, and independent processing structure while applying Anthropic's reduced batch pricing. The official Message Batches documentation states that batch processing reduces input and output token costs by 50%. Requests are processed independently and asynchronously, and each response can be correlated with its original commit through a unique custom\_id.

This workload is a strong batch candidate because it contains many independent requests and does not require an immediate response. Most batches complete within one hour, although processing can continue for up to 24 hours. The pipeline must therefore tolerate asynchronous completion and retrieve the results when processing finishes.

Option A changes the request architecture, increases the risk of context or output-limit failures, and makes individual retry and result association more difficult. Reducing request count does not inherently reduce per-token pricing. Option B improves throughput but concurrency does not change token rates. Option D would reduce cost but violates the explicit requirement to retain the same model tier. The Message Batches API is the only option satisfying every stated constraint.

## Questions 36

---

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

In production, you observe that simple fact-checking queries, such as “In what year was the Paris Climate Agreement signed?”, traverse all four subagents sequentially, consuming more than 40 seconds and significant tokens per query. Complex comparative research benefits from the complete pipeline. Your query distribution is diverse and continues to evolve as users discover new applications.

What is the most effective approach to optimize for varying query complexity?

### OPTIONS

- A. Create a fast path for factual questions that bypasses subagents entirely, routing every other query through the complete pipeline.
- B. Train a query-complexity classifier using labeled historical data to predict the optimal subagent combination, retraining it periodically.
- C. Implement pattern-based routing that classifies queries as single-fact, comparative, or analytical and maps each category to a predefined subagent combination.
- D. Have the coordinator analyze each query and dynamically determine which subagents are required. ✓**

### ANSWER

✓ **Correct answer: D**

### EXPLANATION

Option D allows orchestration effort to scale with the actual request. A simple factual query may require only the web-search agent and a direct coordinator response, whereas a comparative investigation may require web research, document analysis, synthesis, and report generation. Anthropic’s Building Effective AI Agents describes the orchestrator-workers pattern as a central model dynamically identifying subtasks, delegating them, and combining the results. It is specifically appropriate when the required subtasks cannot be predicted reliably in advance. Anthropic’s multi-agent research architecture likewise emphasizes varying the number of agents and tool calls according to task complexity. Option A introduces an inflexible binary decision and still sends every non-factual request through the full pipeline. Option B requires labeled data, ongoing retraining, and reliable definitions of the “optimal” agent combination. Option C is easier to implement but will become brittle as new query types appear. A capable coordinator can examine the requested output, necessary evidence, source requirements, and analytical depth at runtime, then invoke only the specialists that materially contribute to the answer.

## Questions 37

---

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're tasked with adding real-time updates to the application. This could be implemented using WebSockets, Server-Sent Events, or polling, each with different complexity, browser support, and infrastructure requirements.

What's the most effective way to begin this task?

### OPTIONS

- A. Use direct execution to implement polling first, then evaluate whether to upgrade to WebSockets later.
- B. Use direct execution with a prompt asking Claude to analyze all approaches and implement the one it determines is best.
- C. Enter plan mode to explore the architecture, evaluate trade-offs, and present options for team approval before implementing. ✓**
- D. Start direct execution with WebSockets, then refactor if infrastructure issues arise.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

The implementation approach is unresolved and carries architectural consequences. WebSockets, Server-Sent Events, and polling differ in connection lifecycle, bidirectional communication, proxy compatibility, scaling requirements, deployment topology, reconnection behavior, and operational complexity. Claude should therefore inspect the existing application and infrastructure before modifying code.

Anthropic recommends separating exploration and planning from implementation when the approach is uncertain, the work affects multiple files, or the developer is unfamiliar with the relevant architecture. In plan mode, Claude can read files and analyze the system without making changes, produce a detailed proposal, and allow the developer or team to review the plan before execution. ( <https://code.claude.com/docs/en/best-practices> )

Option A commits to polling before determining whether its latency and load characteristics meet the requirement. Option B delegates a consequential architectural decision and immediate implementation to a single execution step without a review checkpoint. Option D prematurely selects the most infrastructure-sensitive option and assumes later refactoring will be inexpensive.

Option C creates the correct sequence: examine the frontend and backend architecture, document requirements, compare alternatives, identify infrastructure constraints, obtain approval, and only then implement the selected design with appropriate tests and observability.

Official references/topics: Plan mode, explore–plan–implement workflow, architectural trade-off analysis, review checkpoints.

## Questions 38

---

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline processes invoices and extracts line items, subtotals, tax amounts, and grand totals. During evaluation, you discover that in 18% of extractions, the sum of extracted line item amounts doesn't match the extracted grand total—sometimes due to OCR errors in the source document, sometimes due to extraction mistakes by the model. Downstream accounting systems reject records with mismatched totals.

What's the most effective approach to improve extraction reliability?

### OPTIONS

- A. Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions.
- B. Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining which extracted values are most likely correct.
- C. Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total.
- D. Add a "calculated\_total" field where the model sums extracted line items alongside a "stated\_total" field. Flag records for human review when values differ. ✓**

### ANSWER

✓ **Correct answer: D**

### EXPLANATION

The pipeline must preserve source evidence while making inconsistencies explicit. Option D records the amount stated on the invoice separately from the total derived from extracted line items. A mismatch then becomes a machine-detectable validation condition rather than an invisible extraction defect.

This approach is superior because it does not silently overwrite source data or ask another model to guess which value is correct. Anthropic's evaluation guidance recommends automated, code-based grading whenever the criterion can be expressed deterministically. Arithmetic reconciliation is precisely such a criterion. ( <https://docs.anthropic.com/en/docs/build-with-claude/develop-tests> ) In production, the summation should preferably be calculated by application code using normalized decimal values, even though the option describes the model populating `calculated_total`. The essential design principle remains the same: preserve `stated_total`, compute an independent total, compare them, and route discrepancies for adjudication.

Option A might improve behavior but cannot resolve genuine OCR corruption and could encourage the model to modify extracted values merely to create mathematical consistency. Option B introduces a second probabilistic judgment without new evidence. Option C is unacceptable for accounting data because it fabricates adjusted amounts and destroys fidelity to the invoice.

The schema should therefore expose both values and attach a validation status or review reason when they differ.

Official references/topics: Deterministic Validation; Human-in-the-Loop Review; Structured Output Design; Source-Fidelity Controls.

## Questions 39

---

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search.

When a user asks the agent to "create a branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

### OPTIONS

- A. Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent. ✓**
- B. The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.
- C. The agent automatically selects the most relevant server based on the request and loads only that server's tools.
- D. You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

MCP allows a Claude application to connect to multiple servers and expose their enabled tools within the same agentic interaction. Anthropic's MCP connector documentation explicitly supports connecting to multiple MCP servers in one request. Once connected, Claude can invoke a server's tool when the user's request corresponds to the capability described by that tool. ( <https://docs.anthropic.com/en/docs/agents-and-tools/mcp-connector> )

In this scenario, the git server can provide the branch-creation operation, the documentation server can locate the relevant links, and the Jira server can update ticket JIRA-123. The agent can coordinate these capabilities without requiring a separate conversational turn that restricts it to only one server. Tool descriptions and schemas tell Claude what each operation does and what inputs it requires.

Option B incorrectly describes sequential capability discovery. Server identity and tool definitions are established through the MCP configuration rather than discovered by querying every server for each operation. Option C is incorrect because connecting several servers does not inherently restrict the agent to a single selected server. Option D imposes a one-server-per-turn limitation that MCP does not require.

Access can still be controlled through allowlists, denylists, permissions, and per-tool configuration. Those controls restrict which tools are enabled; they do not change the fundamental ability to make tools from multiple configured servers available together.

Official references/topics: MCP Server Configuration, Multi-Server Tool Access, Tool Discovery, MCP Tool Permissions.

## Questions 40

---

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

1.5An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~6,000 lines total).

What's the most effective next step for building understanding while managing context constraints?

### OPTIONS

- A. Use Grep to search for “invalidate” and “expire” patterns across all files, then Read only those specific line ranges with minimal surrounding context.
- B. Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.
- C. Use Glob to find files matching common caching patterns ( `cache*.py` , `caching/` ), prioritize the largest files by reading them first, then check smaller files for gaps.
- D. Analyze imports and class hierarchies to identify the base cache class. Read that file to understand the interface, then trace specific invalidation implementations. ✓**

### ANSWER

✓ **Correct answer: D**

### EXPLANATION

The correct objective is to construct an architectural map before consuming the full implementation. Identifying the base cache abstraction, its interface, and the classes that implement or invoke it gives the agent a dependency-guided path through the code. It can then inspect only the invalidation implementations and integration points relevant to the proposed trigger.

This approach protects the context window. Anthropic states that every file read occupies context and that model performance can deteriorate as the window fills. Its Claude Code guidance warns against unbounded investigation that reads large numbers of files and recommends narrowing the exploration or delegating it. ( <https://code.claude.com/docs/en/best-practices> )

Option A is too lexical: searching only for `invalidate` or `expire` can miss event-driven invalidation, overridden methods, cache-key mutation, and generic interface calls. Option B loads approximately 6,000 lines without first establishing relevance. Option C assumes that filename patterns and file size correlate with architectural importance; the largest files may contain incidental code while a small interface defines the entire design.

Option D follows control and type relationships rather than arbitrary file order. After reading the base class, the agent can search for subclasses, imports, construction sites, middleware hooks, and calls to the invalidation contract, progressively expanding only where evidence requires it.

Official references/topics: Context-Efficient Exploration; Dependency-Guided Reading; Architectural Interfaces; Narrowly Scoped Investigation.

## Questions 41

---

In addition to your CI pipeline, your organization has enabled Claude's managed Code Review through the Claude GitHub App on this repository, and reviews run automatically on every pull request. Reviews average 18 findings per pull request. Developer feedback reveals three categories of unwanted noise: (1) style and formatting issues already enforced by your CI linter, (2) findings on automatically generated template code under `src/gen/*`, and (3) rendering-helper patterns that are intentional project conventions but are flagged because they resemble common anti-patterns. Only approximately four findings per pull request are genuine logic bugs. What is the most effective way to reduce this noise while preserving the detection of real issues?

### OPTIONS

- A. Create a `REVIEW.md` file at the repository root containing skip rules for CI-enforced checks and generated files, together with a verification requirement that rendering-related findings cite a specific line demonstrating incorrect behavior. ✓**
- B. Configure separate GitHub Actions workflow files for each code area: one for generated code with findings suppressed, one for rendering code with custom instructions, and one general workflow for everything else.
- C. Add custom review instructions to a GitHub Actions workflow file, using the action's prompt parameter to suppress duplicate lint findings, ignore generated template code, and impose stricter evidence requirements on rendering-related issues.
- D. Add detailed explanations to the project's `CLAUDE.md` describing intentional patterns, stating that CI handles linting, and identifying `src/gen/` as automatically generated code.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

Option A uses the dedicated configuration mechanism for Anthropic's managed Code Review service. The official Code Review documentation states that a root-level `REVIEW.md` is injected into every review agent as the highest-priority instruction block. It can redefine severity, suppress categories already enforced by CI, skip generated paths, and require concrete evidence before particular findings are reported.

The proposed rules map directly to the observed noise. Skipping lint and formatting findings eliminates duplication with deterministic CI checks. Excluding `src/gen/*` prevents comments on machine-generated code. Requiring a specific source line demonstrating incorrect behavior raises the verification threshold for rendering-helper findings without suppressing genuine defects.

Options B and C configure self-hosted GitHub Actions workflows, but the scenario concerns managed Code Review running through the Claude GitHub App on Anthropic's infrastructure. Those workflow prompts do not govern the managed reviewer. Option D supplies useful general project context, but `CLAUDE.md` applies to all Claude Code tasks and has lower review-specific authority. The documentation states that `REVIEW.md` is the stronger control for changing what managed Code Review

flags and how findings are reported. It therefore provides the most targeted solution with the least unnecessary infrastructure.

## Questions 42

---

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent has spent 25 minutes exploring a game engine's rendering subsystem—reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference “typical rendering patterns” rather than the specific VulkanPipeline and FrameGraph classes it discovered earlier.

What's the most effective approach?

### OPTIONS

- A. Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.
- B. Use /clear to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.
- C. Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context. ✓**
- D. Continue in the current context with more targeted prompts referencing the specific classes by name.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

The reference to generic rendering patterns indicates that the current context has become saturated or that important earlier details are no longer receiving sufficient attention. Anthropic documents that performance can degrade as a Claude Code context window fills with conversation history, file contents, and command output; symptoms include forgetting earlier instructions and making less precise responses. ( <https://code.claude.com/docs/en/best-practices> )

Option C preserves the high-value rendering discoveries while isolating the physics investigation. The summary should identify the relevant rendering classes, synchronization boundaries, debug-overlay interfaces, and unresolved integration questions. A physics-focused subagent can then inspect collision and simulation code in its own context without adding dozens of additional files to the already crowded main conversation.

This explicit summary matters because an ordinary named subagent starts with a fresh, isolated context and does not automatically receive the parent's full conversation history or previously read files. Claude constructs a delegation message containing the task context, so the essential rendering findings must be supplied deliberately. ( <https://docs.anthropic.com/en/docs/claude-code/sub-agents> )

Option A omits the integration context needed to guide the physics search. Option B destroys useful prior analysis. Option D keeps expanding an already degraded context and treats the symptom rather than controlling context growth.

Official references/topics: Context-Window Management; Subagent Isolation; Delegation Summaries; Codebase Exploration.

## Questions 43

---

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your pipeline reviews every pull request using a single API call with a static prompt containing the diff and the full text of each changed file. Unchanged files are not included. Developers report that reviews consistently miss cross-file bugs—for example, a pull request renames a function's parameters, but the review does not identify callers in unchanged files that still use the old argument order.

Evaluation shows that cross-file bugs account for 35% of production incidents originating from reviewed pull requests.

What is the most effective change to the review design?

### OPTIONS

- A. Build a static dependency graph and include every file located within two dependency hops of a changed file.
- B. Add instructions asking the model to list external references and reason step by step about how each change could affect unseen callers.
- C. Redesign the review as a turn-limited agentic task that can read files and search the repository, following references to verify cross-file findings. ✓**
- D. Run separate review passes for each changed file with its direct dependants, and then aggregate and deduplicate the findings through a final consolidation pass.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

Option C gives the reviewer access to the evidence currently missing from its prompt. An agentic review can use Grep, Glob, Read, language-server tools, and test commands to locate callers, follow imports, inspect type definitions, and verify whether a suspected compatibility issue actually exists. A turn limit controls cost while still allowing targeted exploration.

Anthropic's context-engineering guidance recommends just-in-time retrieval: agents should retain lightweight references and dynamically load the information needed for the current task instead of preloading a large fixed context. Anthropic has also reported that code-review performance improves when the necessary repositories are available for gathering complete context. Option A may load many irrelevant files while still missing reflection, generated interfaces, dependency injection, or indirect call paths beyond the arbitrary two-hop boundary. Option B cannot reason reliably about code that the model cannot inspect. Option D improves coverage but fragments repository-wide relationships and may duplicate substantial work. An agentic loop can begin with the diff, identify uncertain cross-file effects, search only relevant paths, and report findings only after locating supporting code. Anthropic context-engineering guidance , Claude Code quality report

## Questions 44

---

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The coordinator agent has AgentDefinition objects configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice that the coordinator correctly reasons about when to delegate—it generates messages such as, “I’ll ask the web-search agent to find sources on this topic”—but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors.

What is the most likely cause?

### OPTIONS

- A. The AgentDefinition objects are configured correctly, but the coordinator’s system prompt does not explicitly list the available subagent types, preventing the model from knowing that they can be invoked.
- B. Subagent context isolation means task descriptions from the coordinator do not automatically reach subagents; you must configure explicit context forwarding in ClaudeAgentOptions.
- C. The coordinator’s allowedTools configuration does not include Agent—formerly named Task—so it cannot invoke the tool required to spawn subagents. ✓**
- D. The coordinator’s max\_tokens setting is too low, causing the subagent tool invocation to be truncated before the subagent type can be specified.

### ANSWER

✓ **Correct answer: C**

### EXPLANATION

Option C matches the distinction between reasoning about delegation and executing it. Defining subagents makes their descriptions available for selection, but the coordinator must still invoke the SDK’s subagent-spawning tool. Current Claude Agent SDK documentation calls this the Agent tool; Task was its earlier name and remains relevant to older SDK configurations. Anthropic’s Subagents in the SDK documentation instructs developers to include Agent in allowedTools so subagent invocations are approved automatically. Without that permission, an invocation can fall through to a permission callback or be denied under a non-interactive permission mode. Option A is unlikely because the configured subagent descriptions already tell Claude when each agent should be selected, although explicit prompting can improve invocation reliability. Option B misstates context isolation: context must be included in the spawning prompt, but that issue occurs after an invocation is attempted and does not explain the absence of all subagent executions. Option D would normally produce truncation evidence or incomplete output rather than consistent verbal promises with no tool call. The configuration should therefore permit Agent, explicitly request delegation where necessary, and log subagent invocation events.

## Questions 45

---

The web-search agent has gathered several relevant sources for a research topic. The document-analysis agent now needs to examine those sources. How does information typically flow between these two specialized subagents?

### OPTIONS

- A. The coordinator receives the web-search agent's output and includes the relevant sources and findings in the prompt used to invoke the document-analysis agent. ✓**
- B. The web-search agent directly invokes the document-analysis agent and passes the discovered sources as parameters.
- C. The agents communicate through an event-driven message queue, with the document-analysis agent subscribing to web-search completion events.
- D. Both agents automatically access a shared memory store in which the web-search agent writes its findings.

### ANSWER

✓ **Correct answer: A**

### EXPLANATION

Option A matches the standard coordinator-managed subagent model. The coordinator invokes the web-search agent, receives its final result, selects the useful URLs or source records, and then supplies them explicitly when invoking the document-analysis agent. This keeps task ownership, provenance, and execution order visible to the coordinator.

The Claude Agent SDK subagent documentation states that a subagent begins with a fresh context window and does not inherit the parent's conversation history or previous tool results. The information passed through the Agent tool's prompt is the primary parent-to-subagent context channel. Consequently, the analysis prompt must include the source URLs, documents, retrieval notes, and any questions the analyzer must answer.

Option B requires nested delegation that is neither stated nor necessary and reduces centralized observability. Options C and D describe valid custom architectures only if the application has deliberately implemented a queue or shared memory system; neither is automatic Agent SDK behavior. Coordinator-mediated handoff is therefore the expected approach. For reliability, the handoff should use structured source records rather than an informal prose statement, ensuring that the analyzer receives identifiers, provenance, relevant excerpts, and analysis objectives.